Intermediate (temperature and cost, side by side) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and a worked example

Hint 1 — The idea, and the exact pieces

Part 1: the key idea to predict *before* you run anything: not every prompt is equally sensitive to temperature. A prompt with essentially one correct answer ("what's 7 times 8?") will barely change even at high temperature, because the model is extremely confident about the next tokens regardless of the sampling setting. A genuinely open-ended prompt ("write a short story about a lighthouse") will visibly change across runs at `temperature=1`, because many different continuations are all reasonably likely.

Part 2: the formula is `(input_tokens × input_price_per_token) + (output_tokens × output_price_per_token)`. Prices are usually quoted per 1,000 or per 1,000,000 tokens on a pricing page — convert to a per-token rate first, or keep everything in the same unit (e.g. "per million tokens") throughout the calculation so you don't mix units.

The exact pieces:

- **A good test prompt** — genuinely open-ended but short enough to compare 6 answers side by side quickly, like "give me a one-sentence tagline for a coffee shop."
- **Predict specifically** — will all 3 `temperature=0` answers be word-for-word identical, or just very similar? (In practice: usually very similar, not always byte-identical — there's some randomness elsewhere in the serving system even at `temperature=0`.)
- **Units, made explicit** — if the price is "$0.15 per 1M tokens," convert it: `0.15 / 1_000_000` dollars per token, or keep the token counts in millions too.

Write your Part 1 prediction and your Part 2 formula (with real numbers plugged in) before checking Hint 2.

One thing worth knowing before you predict: `temperature=0` reduces variation but doesn't guarantee an identical answer every single time — there's genuine floating-point non-determinism in how modern GPU serving batches multiple requests together, which slightly changes rounding. "Deterministic" here means "extremely consistent," not "provably identical."

Hint 2 — The plan, and a worked example

```
part 1:
    prompt = "give me a one-sentence tagline for a coffee shop"
    for temp in [0, 1]:
        for i in range(3):
            call the model with temperature=temp
            print the answer

    compare the 3 temp=0 answers to each other
    (expect: very close, maybe identical)
    compare the 3 temp=1 answers to each other
    (expect: noticeably different wording/ideas)

part 2:
    cost = (input_tokens / 1_000_000) * input_rate_per_million \
         + (output_tokens / 1_000_000) * output_rate_per_million
```

The same worked example, shown as a small formula you can reuse for any call:

```
input_rate_per_million  = 0.15   # dollars — look up the real current rate
output_rate_per_million = 0.60   # dollars

input_tokens  = 2000
output_tokens = 500

cost = (input_tokens / 1_000_000) * input_rate_per_million \
     + (output_tokens / 1_000_000) * output_rate_per_million
```

Plug in real current numbers for a model of your choice from OpenAI's pricing page, and compute the cost of this document's own example call (2,000 input / 500 output tokens). Then write out your Part 1 temperature predictions in full sentences — "I expect the temp=0 answers to be ___ because ___" — then compare against the full Solution.

Full solution: Show me the solution